THE QASKILLS.SH PLAYBOOK

The Claude Code QA Playbook

10+ skills to turn your AI coding agent into a disciplined QA engineer, with copy-paste prompts, real code, and the traps to avoid.

By Pramod Dutta, The Testing Academy · QASkills.sh · 2026 edition

Why this playbook exists

AI coding agents like Claude Code will happily write tests. Left unguided, they write the *wrong* tests: happy-path only, brittle CSS selectors, assertion-free snapshots, tautologies that pass without proving anything. The difference between an agent that ships quality and one that ships noise is not the model. It is the **skill** you give it: a reusable SKILL.md file that encodes how a senior QA engineer actually works.

This playbook walks through 10+ QA skills you can hand your agent today. Each one is a real, installable skill from QASkills.sh, the open registry of 400+ QA skills. For five of them, you already have the full SKILL.md file in the ZIP that came with this download.

> **How to use a skill:** drop the SKILL.md folder into `.claude/skills/` (project) or `~/.claude/skills/` (global), or run `npx qaskills add <skill>`. The agent loads it automatically when the task matches.

The 10+ skills

1 Playwright E2E

The most-installed QA skill. Teaches the agent resilient end-to-end testing: role-based locators (`getByRole`, `getByLabel`) over brittle CSS/XPath, Playwright's auto-waiting instead of `waitForTimeout`, the Page Object Model, and fixtures for setup. The single biggest quality lever for any web app.

```
// What a skilled agent writes: intent-based, auto-waiting
await page.getByRole('button', { name: 'Add to cart' }).click();
await expect(page.getByTestId('cart-total')).toHaveText('$29.00');
```

2 Unit Test Generation

Turns "write tests for this function" into high-signal coverage instead of line-padding. Enforces boundary values, the full error contract, mocking only at architectural boundaries, and a break-the-code check so every test provably bites. Pairs with mutation testing for critical modules.

3 PR Test Coverage Review

Makes the agent a reviewer: it maps each change in a diff to the tests it obligates (new branch = both sides tested; bug fix = a regression test that reproduces the bug), then flags tautological or over-mocked tests. Stops untested error

paths from shipping on money flows.

4 DeepEval LLM Evaluation

If your product has an AI feature, this is how you test it. Pytest-style evals with metrics for hallucination, answer relevancy, and faithfulness, run as CI gates against a versioned golden dataset. Evals are unit tests for LLM output, not a dashboard afterthought.

```
from deepeval.metrics import FaithfulnessMetric
assert_test(case, [FaithfulnessMetric(threshold=0.9)])
```

5 Jira QA Workflows

The manual-QA companion. Bug templates optimized for a developer's first 60 seconds, JQL queries every tester should know, triage rituals, and quality dashboards that answer real questions (escape rate, reopen rate, release readiness) instead of vanity counts.

6 API Testing

Request/response coverage done right: status and schema assertions, auth flows, negative cases, and contract testing so a renamed field cannot silently break a consumer. Covers Postman/Newman, REST-assured, and code-first HTTP testing.

7 Visual Regression (Percy)

Catches the bugs functional tests cannot see: layout breaks, overlap, contrast. Teaches snapshot-per-state (not per-page), dynamic-content stabilization, and review discipline so baselines do not rot into rubber-stamps.

8 Self-Healing Locators

The 2026 answer to selector churn. Role-based, intent-first locators plus agent-driven repair: when a selector breaks, the agent probes the live DOM and fixes it, instead of a human updating forty tests after a class rename.

9 Flaky Test Detection

Turns "the suite is flaky" into a tracked, fixable metric. Quarantine strategy, retry-then-analyze, and root-causing race conditions and timing dependencies so failure signal stays trustworthy.

10 Test Metrics & KPIs

What a QA lead actually reports: escape rate, flake rate, diff coverage, MTTR, each paired with its anti-gaming partner, plus a release-readiness scorecard filled by queries, not opinions.

+ And 390 more

Kafka event testing, contract testing (Pact), accessibility (axe/pa11y), load testing (k6), BDD, mutation testing, RAG evaluation (Ragas), promptfoo red-teaming, and hundreds more, each a real SKILL.md at QASkills.sh.

The workflow that ties them together

Skills compose. A mature agent QA loop looks like this:

Stage	Skill	What the agent does
Author	Playwright E2E, Unit Test Gen	Write tests from intent, not from the current implementation

Evaluate	DeepEval / Ragas	Gate AI features on measured quality
Review	PR Test Coverage Review	Block merges with untested branches
Maintain	Self-Healing, Flaky Detection	Repair selectors, quarantine flakes
Report	Test Metrics, Jira Workflows	Escape rate, release readiness, triage

> **The one rule that matters most:** a generated test you have not watched fail is unverified. Make the agent break the code and confirm the test catches it, every time.

Your 5-skill starter pack is in the ZIP.
Install the full catalog of 400+ QA skills, read 800+ QA guides, and see the leaderboard at qaskills.sh.
Install any skill in one line: `npx qaskills add <skill>`